Instruction Set Architecture (ISA)

Personal Handwritten Notes — Typed & Organized Version

---

1. What is ISA?

The **language that software uses to communicate with the CPU hardware**.

- Serves as an **interface between software and hardware**.
- Consists of information regarding the **view of the architecture**:
- Registers
- Address buses
- Data buses
- Memory models
- ...and the **complete instruction set**.
- Many ISAs are **not specific to a particular computer architecture (hardware design)** — the same ISA can run on many different hardware implementations.

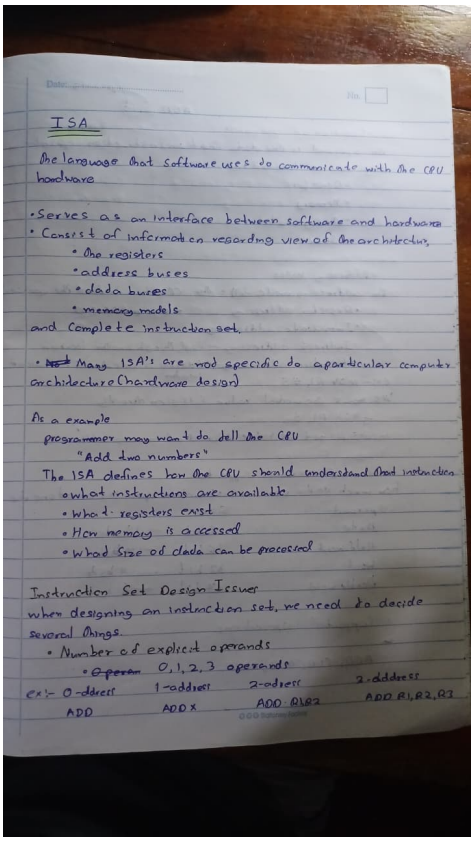
Example: A programmer may want to tell the CPU “Add two numbers.” The ISA defines how the CPU should understand that instruction:

- What instructions are available
- What registers exist
- How memory is accessed
- What size of data can be processed

Instruction Set Design Issues

When designing an instruction set, we need to decide several things — starting with the **number of explicit operands**:

Operands	Type	Example
0	0-address	`ADD`
1	1-address	`ADD X`
2	2-address	`ADD R1, R2`
3	3-address	`ADD R1, R2, R3`



ISA

the language that software uses to communicate with the CPU hardware

- Serves as an interface between software and hardware
- Consest of information regarding view of the architecture,
  - the registers
  - address buses
  - data buses
  - memory models

and complete instruction set.

- ~~Not~~ Many ISA's are not specific to a particular computer architecture (hardware design)

As a example

programmer may want to tell the CPU

"Add two numbers"

The ISA defines how the CPU should understand that instruction

- what instructions are available
- what registers exist
- How memory is accessed
- what size of data can be processed

Instruction Set Design Issues

when designing an instruction set, we need to decide several things

- Number of explicit operands
  - ~~0 operan~~ 0, 1, 2, 3 operands

ex:-	0-address	1-address	2-address	3-address
	ADD	ADD x	ADD R1,R2	ADD R1,R2,R3

Original handwritten page — ISA definition & instruction set design issues

2. Location of Operands & Addressing Modes

An **operand** is the data that an instruction operates on. Operands can be located in:

- Registers
- Memory
- Accumulator
- Stack

Addressing Modes

An addressing mode tells the CPU **how to find the operand**:

- **Register addressing** — operand is in a register.
- **Memory addressing** — operand is in memory.
- **Immediate addressing** — the actual value is given directly in the instruction.
- **Indirect addressing** — sent through another location (a pointer to the operand).
- **Relative addressing** — address is calculated relative to the current location.

Example — Immediate vs. Direct/Register addressing:

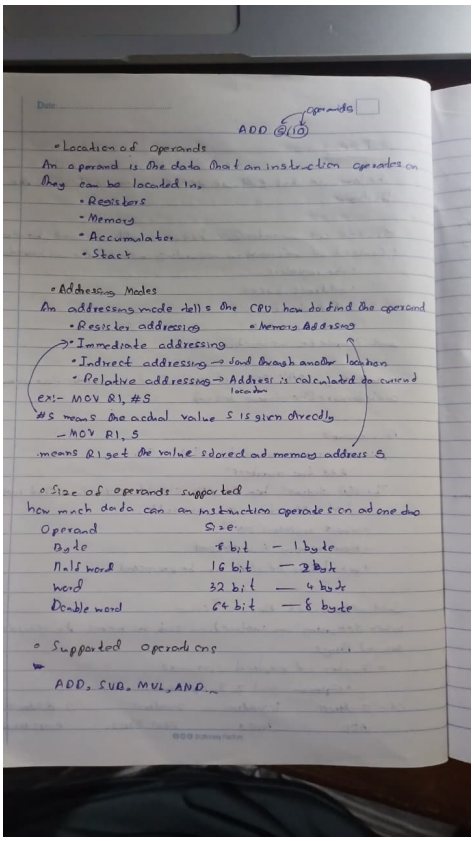
```
MOV R1, #5 ; # means the actual value 5 is given directly MOV R1, 5 ; R1 gets the value stored at memory address 5
```

Size of Operands Supported

How much data can an instruction operate on at one time:

Operand	Size (bits)	Size (bytes)
Byte	8 bit	1 byte
Half word	16 bit	2 byte
Word	32 bit	4 byte
Double word	64 bit	8 byte

Supported operations (examples): **ADD, SUB, MUL, AND, OR, CMP, MOVE, JMP**, etc.



Original handwritten page — addressing modes & operand sizes

3. CPU Architecture Types & General Purpose Registers

Summary of architecture styles by number of addresses:

Architecture	Example	Main Idea / Data Path
0 Address / stack	`PUSH X, PUSH Y, ADD, POP Z`	Stack → ALU
1 address / ACC	`LOAD X, ADD Y, STORE Z`	ACC + Mem → ALU → ACC
2 address / Reg-Mem	`LOAD R2,X, ADD R2,Y`	Reg + Mem → ALU → Reg
2/3 address / Mem-Mem	`ADD X,Y,Z`	Mem + Mem → ALU → Mem
3 address / Reg-Reg	`ADD R3,R1,R2`	Reg + Reg → ALU → Reg

General Purpose Registers (GPRs)

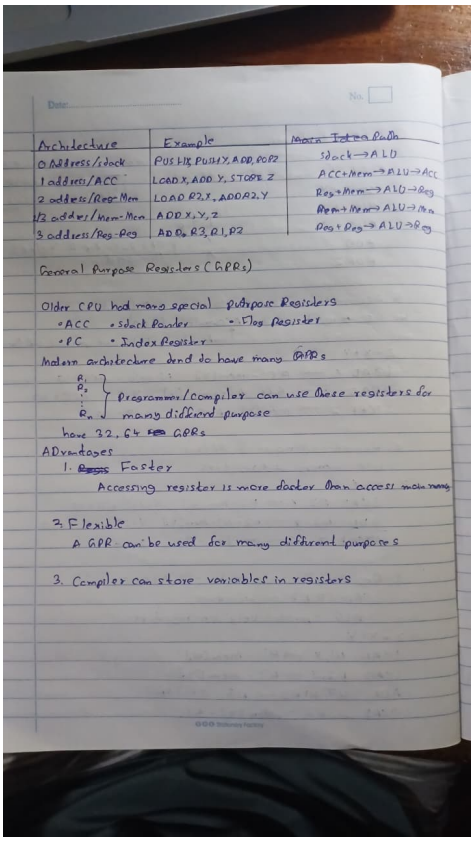
Older CPUs had many **special-purpose registers**, each with one job:

- Accumulator (ACC)
- Program Counter (PC)
- Stack Pointer
- Flag Register
- Index Register

Modern architectures instead have many **GPRs** (R1, R2, ... Rn) that the programmer or compiler can use for many different purposes — commonly **32 or 64 GPRs**.

Advantages of GPRs

- **Faster** — accessing a register is much faster than accessing main memory.
- **Flexible** — a GPR can be used for many different purposes.
- The **compiler** can store variables in registers.



Architecture	Example	~~Main Instruction~~
0 Address / Stack	PUSH X, PUSH Y, ADD, POP Z	Stack → ALU
1 address / ACC	LOAD X, ADD Y, STORE Z	ACC + Mem → ALU → ACC
2 address / Reg-Mem	LOAD R2, X, ADD R2, Y	Reg + Mem → ALU → Reg
3 address / Mem-Mem	ADD X, Y, Z	Mem + Mem → ALU → Mem
3 address / Reg-Reg	ADD R3, R1, R2	Reg + Reg → ALU → Reg

General Purpose Registers (GPRs)

Older CPU had many special purpose Registers
- ACC
- Stack Pointer
- Flag Register
- PC
- Index Register

Modern architecture tend to have many GPRs

R_1, R_2, ..., R_n } Programmer / compiler can use these registers for many different purpose

have 32, 64 ~~GPRs~~ GPRs

ADvantages

1. ~~Easy~~ Faster
   Accessing register is more faster than access main memory
2. Flexible
   A GPR can be used for many different purposes
3. Compiler can store variables in registers

Original handwritten page — architecture summary table & GPRs

4. CPU Architectures — Worked Example: Z = X + Y

The special area is a fast, small memory used to store data. Comparing **LOAD place** → **load remove value into register/ACC** and **STORE place** → **store value from register/ACC to memory** across the four architecture styles:

A. Stack-Based (0-address)

```
Z = X + Y PUSH X PUSH Y ADD POP Z
```

The stack has a **Top (TOS)**. The CPU takes the top 2 values off the stack, then adds them.

B. Accumulator-Based Machine (1-address)

We use a register called the **Accumulator (ACC)**. It automatically holds one operand and the result.

```
Z = X + Y LOAD X ; ACC = Mem[X] (if X=10, ACC=10) ADD Y ; ACC = ACC + Mem[Y] (if
Y=20, ACC=30) STORE Z ; Mem[Z] = ACC (Z=30)
```

C. Register-Memory Machine (2-address)

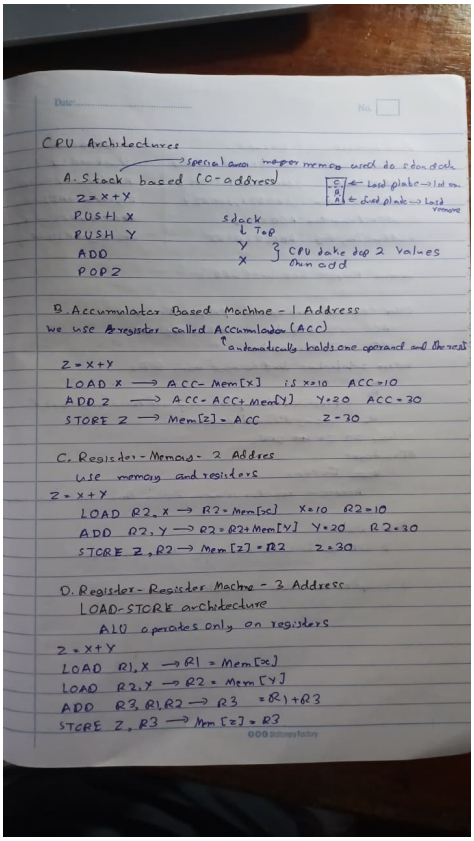
Uses memory and registers together.

```
Z = X + Y LOAD R2, X ; R2 = Mem[X] (X=10, R2=10) ADD R2, Y ; R2 = R2 + Mem[Y] (Y=20,
R2=30) STORE Z, R2 ; Mem[Z] = R2 (Z=30)
```

D. Register-Register Machine (3-address) — Load-Store Architecture

The ALU operates **only on registers**.

```
Z = X + Y LOAD R1, X ; R1 = Mem[X] LOAD R2, Y ; R2 = Mem[Y] ADD R3, R1, R2 ; R3 = R1
+ R2 STORE Z, R3 ; Mem[Z] = R3
```



Original handwritten page — CPU architecture worked examples

Quick Summary

- ISA = the language software uses to talk to CPU hardware; it's the interface between the two.
- Design issues: number of operands, operand location, addressing modes, operand size, supported operations.
- Addressing modes tell the CPU how to find an operand: register, memory, immediate, indirect, relative.
- Architecture styles: 0-address (stack), 1-address (accumulator), 2-address (register-memory), 3-address (register-register / load-store).
- Modern CPUs use many GPRs instead of few special-purpose registers — faster, flexible, and compiler-friendly.